

CoopInfer: 边端协同推理调度原型

面向 DAG 切分、约束求解与时序可视化的桌面验证工具

CoopInfer 项目

2026 年 6 月 24 日

- 具身智能系统常由感知、融合、控制等算子组成，并形成 DAG。
- 部分算子必须固定在机器人端侧设备，部分算子可以卸载到边侧主机。
- 真实调度不仅是图切分：
 - 跨设备边会产生网络传输代价；
 - 端侧和边侧都有有限执行队列；
 - 端到端时延上限会直接排除不可行方案。
- CoopInfer 用于快速编辑、求解、验证并可视化这类调度方案。

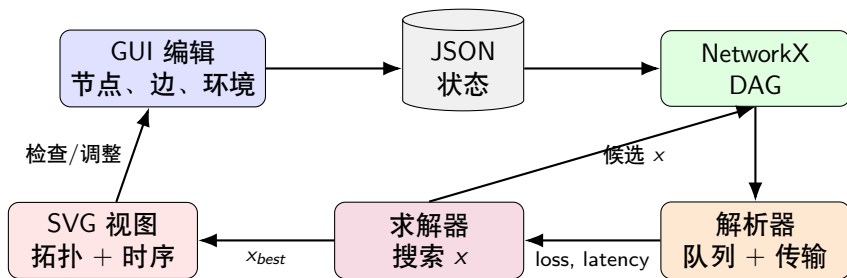
输入

- DAG 节点及端侧/边侧执行耗时。
- 边数据量。
- 带宽、固定时延和优化权重。
- 串行网络模型以及可选传输批处理。
- 可选平均端到端时延上限和最大单帧时延上限。

输出

- 节点放置决策： $x_i = 0$ 表示端侧， $x_i = 1$ 表示边侧。
- 端到端时延和端侧算力利用率。
- SVG 拓扑图和 profiler 风格时序图。

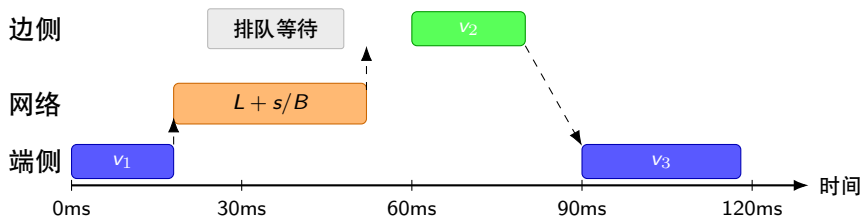
原型闭环



原型把编辑、求解、物理推演和可视化放在一个可复现实验闭环中。

- 内存中使用 `networkx.DiGraph` 存储拓扑。
- 节点属性：
 - `id`, `name`, `c_dev`, `c_host`, `fixed_dev`, `x`
 - 可选输入频率字段: `source_period_ms`, `source_phase_ms`
- 边属性：
 - `source`, `target`, `size`, 单位为 MB。
- 环境参数：
 - `bandwidth`, `latency`, `weight_avg_latency`, `weight_max_latency`, `weight_device_utilization`
 - `latency_limit`, `max_frame_latency_limit`, `batch_transfers`
- JSON 导入/导出保证实验可复现、可共享。

考虑队列的执行过程



节点必须同时满足“数据已就绪”和“目标设备队列空闲”两个条件才能开始执行。

内层解析器：数据依赖与资源排队

对 DAG 按拓扑序遍历。对节点 i :

$$DataReady_i = \max \left(release_i(f), \max_{j \in pred(i)} \left[F_j + \mathbb{I}(x_j \neq x_i) \left(L + \frac{S_{ji}}{B} \right) \right] \right)$$

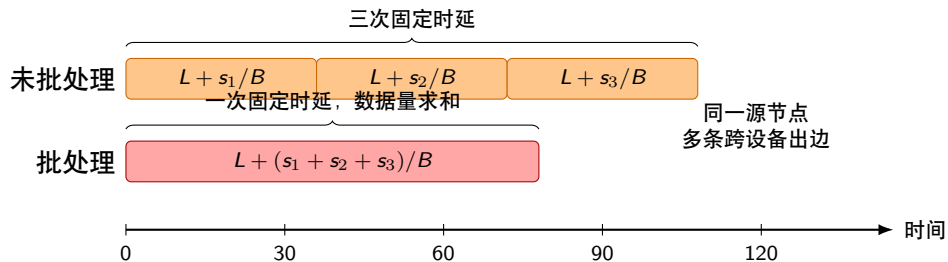
输入源节点可使用 $release_i(f) = phase_i + f \cdot period_i$ 表示第 f 帧的生成时刻。源节点是零耗时的外部输入事件，不占用端侧或边侧计算队列；后续帧输入可以在前一帧下游计算尚未完成时继续按周期到达。数据就绪后，还要等待目标设备队列空闲：

$$S_i = \begin{cases} \max(DataReady_i, time_dev_ready), & x_i = 0 \\ \max(DataReady_i, time_host_ready), & x_i = 1 \end{cases}$$

$$F_i = S_i + (1 - x_i)c_i^{dev} + x_i c_i^{host}$$

端到端时延从第一个非输入计算或传输事件开始，到最后一个非输入输出节点完成结束。输入源释放会推迟下游工作，但不计入时延窗口内部。

网络批处理示意



批处理减少重复固定时延，但不会让网络传输发生重叠。

网络模型：串行传输与批处理

- 所有跨设备传输共享一条网络工作队列：

$$time_network_ready$$

- 传输必须等待源节点完成，并且等待网络队列空闲，因此不同传输不会重叠。
- 未开启批处理时，每条跨设备边独立计费：

$$T_{edge} = L + \frac{s}{B}$$

- 开启 `batch_transfers=true` 后，同一个源节点的多条跨设备出边会作为一次网络事务发送：

$$T_{batch} = L + \frac{\sum_k s_k}{B}$$

- 这样既保持网络非重叠，又避免连续出边重复支付固定时延。

优化目标与可行性约束

求解器最小化加权目标：

$$J = w_{avg}L_{avg} + w_{max}L_{max} + w_{util}L_{util}$$

其中

$$L_{avg} = \frac{\tau}{\max(scale_{avg}, 1)}, \quad L_{max} = \frac{\tau_{frame}^{max}}{\max(scale_{max}, 1)}, \quad L_{util} = 1 - \frac{device_active_time}{pipeline_work_span}$$

时延归一化尺度来自全端侧/全边侧基线。

时延上限

正数上限是硬约束：

$$\tau \leq latency_limit, \quad \tau_{frame}^{max} \leq max_frame_latency_limit$$

取值为 0 表示关闭对应约束；超过任一启用上限的候选方案会在目标函数比较前被直接拒绝。

求解模式

模式	适用场景	行为
Auto	默认模式	自由节点 ≤ 12 时枚举，否则随机搜索
Enumerate	小规模 DAG	遍历所有方案，返回全局最优可行解
Random Search	较大 DAG	随机扰动放置方案，保留最佳可行候选
Simulated Annealing	跳出局部最优	按温度接受部分更差移动

所有模式都会强制固定端侧节点，并执行平均端到端和最大单帧时延上限过滤。

GUI 使用流程

- ① 编辑节点、名称、计算耗时和固定端侧标记。
- ② 编辑边和传输数据量。
- ③ 配置带宽、固定时延、网络批处理、优化权重、求解模式、迭代次数和时延上限。
- ④ 点击 **Solve**。
- ⑤ 查看性能看板、拓扑放置和 profiler 风格时序图。

系统会校验重名节点、非法边端点以及非 DAG 拓扑。

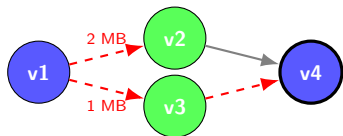
拓扑 SVG

- 配置 DAG 在求解前展示数据量以及本地/跨设备边样式。
- 求解后的展开流水线使用高对比填充色表示帧编号。
- 蓝色轮廓表示端侧，绿色轮廓表示边侧。
- 加粗或虚线源节点样式表示固定端侧和输入源事件。
- FIFO、传输、本地依赖和释放线都跟随源帧的同一颜色。

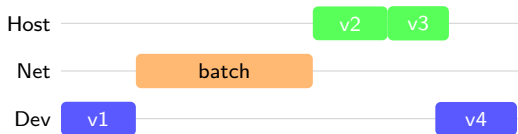
时序 SVG

- Input、Host、Transfer、Device 四条泳道。
- 算子条由开始/完成时间决定，并使用与拓扑图一致的帧填充色。
- 传输条来自解析器的真实传输记录，可展示串行传输和批处理传输。
- 输入释放点、周期跨度和垂直释放/依赖线都使用对应帧颜色。
- 使用浏览器原生滚动和缩放控件。

拓扑图与时序图读法

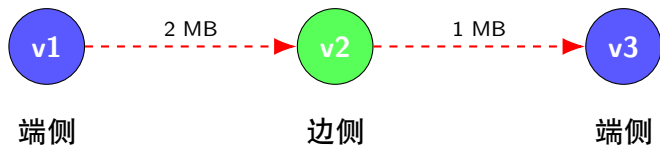


拓扑图：颜色表示放置位置，边样式表示通信关系



时序图：条形位置来自解析器计算出的开始/完成时间

示例 DAG



解析器输出的是考虑设备队列的真实调度，而不是理想无限并行关键路径。

当前技术栈

- Python 3.9+。
- PyQt6 负责桌面控件和布局。
- PyQt6-WebEngine 负责 SVG 拓扑图和时序图渲染。
- NetworkX 负责 DAG 存储、拓扑排序和环路检测。
- C++/pybind11 原生核心负责调度评估、目标函数计算和求解搜索。
- Pytest 覆盖解析器、JSON IO、求解模式和时延上限可行性。
- 启动命令: `python -m coopinfer` 或 `python -m coopinfer.app`。

- 增加求解诊断：被拒绝候选、约束裕量、收敛历史。
- 支持拓扑图和时序图导出为 SVG/PNG 报告。
- 增加带宽/时延参数扫描的批量评估。
- 为大规模 DAG 引入更高级的调度算法。
- 在时序图中显式区分队列等待、数据就绪和传输阶段。
- 探索比拓扑源节点 FIFO 更灵活的网络重排策略。

Q&A